

Cybersecurity Internship Report

Week 1 – Security Assessment

Name: Abdul Ahad

Internship: Cybersecurity Analyst Internship

Week: 1 – Security Assessment

Objective

The objective of this week's task was to analyze a web-based User Management System, identify common security vulnerabilities, and document possible security improvements using manual testing, developer tools and OWASP ZAP.

Application Setup

React

Node.js

MySQL

XAMPP

phpMyAdmin

Visual Studio Code.

OWASP ZAP

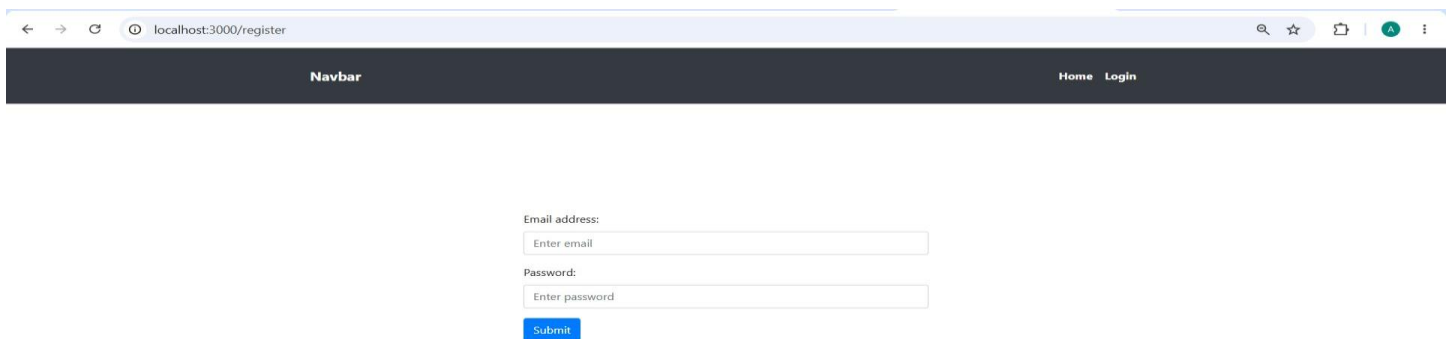
Java JDK 25 LTS

Git

GitHub

Google Developer Tools

Application running (Login page)



← → ↻ 📄 localhost:3000/register 🔍 ☆ 📄 | 🟢 ⋮

Navbar Home Login

Email address:

Password:

Security Testing

1. Cross-Site Scripting (XSS)

Test Performed

```
<script>alert('XSS')</script>
```

Result

The payload was rejected because the email field only accepts valid email addresses. No JavaScript was executed.

Status: Not Vulnerable during testing

XSS Test Result

The screenshot shows a web browser at the URL `localhost:3000/register`. The page has a dark navbar with "Navbar", "Home", and "Login" links. The main content area contains a registration form with the label "Email address:". The input field contains the payload `<script>alert('XSS')</script>`. A red error message box is displayed below the input, stating: "Please include an '@' in the email address. '<script>alert('XSS')</script>' is missing an '@'." Below the error message is a password field with masked characters (dots) and a blue "Submit" button.

2. SQL Injection

Test Performed

`admin' OR '1'='1`

Result

The login was not bypassed. The application rejected the input and authentication failed.

Status: SQL Injection not successful

SQL Injection Test Result

The screenshot shows the same registration page as above, but with the payload `admin' OR '1'='1` entered in the "Email address:" field. The red error message box now states: "Please include an '@' in the email address. 'admin' OR '1'='1' is missing an '@'." The "Submit" button is visible, and a link "create a account" is at the bottom.

3. Password Storage

The database was inspected using phpMyAdmin.

Passwords were stored as **bcrypt hashes** instead of plain text.

Status: Secure Password Hashing Implemented

Database showing hashed passwords

← T →		▼	id	email	password	date
☐	Edit Copy Delete		1	somu@gmail.com	\$2b\$10\$xy6mo4clHl8tHxZ12F3xf.i8CmqNsFUEkxXdCBTzzQW...	2021-06-26 11:37:39
☐	Edit Copy Delete		2	somuu@gmail.com	\$2b\$10\$GVcA24UNA.JV3dU5zgFCRuU4s9bn0nUhaYVZRu.JVzi...	2021-06-26 11:38:13
☐	Edit Copy Delete		3	ram@gmail.com	\$2b\$10\$0bp6mwJ2aJnTh3LuGpJtTuMSVqw1WNQJbK7EL6A8R59...	2021-06-26 11:46:15
☐	Edit Copy Delete		4	atanuj383@gmail.com	\$2b\$10\$NN2z7zs6loysol8OEa1xelBhuM6Crrh/woHyYHzLJe...	2021-06-26 12:22:18
☐	Edit Copy Delete		5	atanuj5@gmail.com	\$2b\$10\$fKTJbChGRpZLgRkXS6moA.zQATJ8jVoBU66wV57vkeE...	2021-06-26 12:27:05
☐	Edit Copy Delete		6	atanuj55@gmail.com	\$2b\$10\$nnbEitRzcfPw28a7XGc8e00Zvm38sob6l8Bv8nyxOV...	2021-06-26 12:27:55
☐	Edit Copy Delete		7	rajuahamed.kh@gmail.com	\$2b\$10\$DrNNqqSNibwoH/1fPW9t.9LUgbY28kge4FLcH0BYKh...	2021-06-26 14:45:36
☐	Edit Copy Delete		8	kkk@gmail.com	\$2b\$10\$WvfdXiguQbF8zoSfXy7qGeneV0XGSdglMGriRppaJgi...	2021-06-26 15:02:13
☐	Edit Copy Delete		9	samrat1@gmail.com	\$2b\$10\$dXEjGzCNaPIRybHMRnK06.cbwnA6vTkSjld2q9xKLbD...	2021-06-26 17:45:07

4. Browser Inspection

Google Chrome Developer Tools were used to inspect the login request.

Findings

- Login uses the **POST** method.
- Password is sent in the request body.
- No JWT token is returned after login.
- Session authentication is used through a cookie.

5. OWASP ZAP Scan

An automated vulnerability assessment was performed using OWASP ZAP.

The following issues were identified:

Vulnerability	Brief Description
Content Security Policy (CSP) Header Not Set	The application does not define a Content Security Policy, increasing the risk of certain client-side attacks.
Missing Anti-clickjacking Header	The application can potentially be embedded inside another webpage.
HTTP Only Site	The application uses HTTP instead of HTTPS.
X-Content-Type-Options Header Missing	Browsers may perform MIME-type sniffing, which can introduce security risks.
Server Leaks X-Powered-By Header	The server reveals that it is running Express.js.
Subresource Integrity Missing	External resources are loaded without integrity verification.

OWASP ZAP Scan Results

The screenshot displays the OWASP ZAP web application security scanner interface. The top toolbar includes buttons for History, Search, Alerts, Active Scan, and Output. The left sidebar shows a tree view of Alerts (10), listing various vulnerabilities such as CSP: Failure to Define Directive with No Fallback (3), Content Security Policy (CSP) Header Not Set, HTTP Only Site, Missing Anti-clickjacking Header, Sub Resource Integrity Attribute Missing (4), Cross-Domain JavaScript Source File Inclusion (3), Server Leaks Information via "X-Powered-By" HTTP Response Head, X-Content-Type-Options Header Missing (Systemic), Information Disclosure - Suspicious Comments (3), and Modern Web Application. The main pane on the right provides instructions on how to add and edit alerts. The bottom status bar shows the number of alerts (0) and the main proxy address (localhost:8080).

History Search Alerts Active Scan Output +

Alerts (10)

- > CSP: Failure to Define Directive with No Fallback (3)
- > Content Security Policy (CSP) Header Not Set
- > HTTP Only Site
- > Missing Anti-clickjacking Header
- > Sub Resource Integrity Attribute Missing (4)
- > Cross-Domain JavaScript Source File Inclusion (3)
- > Server Leaks Information via "X-Powered-By" HTTP Response Head
- > X-Content-Type-Options Header Missing (Systemic)
- > Information Disclosure - Suspicious Comments (3)
- > Modern Web Application

Full details of any selected alert will be displayed here.

You can manually add alerts by right clicking on the relevant line in the history and selecting 'Add alert'.

You can also edit existing alerts by double clicking on them.

Alerts 0 5 3 2 Main Proxy: localhost:8080

Type here to search

After exploring through the sign up, login and home page with developers tools, I found these vulnerabilities.

Vulnerabilities Found

- Missing Content Security Policy (CSP)
- Missing Anti-clickjacking Header
- Missing X-Content-Type-Options Header
- Server Information Disclosure (`X-Powered-By`)
- HTTP used instead of HTTPS (Development Environment)
- JWT Authentication not implemented

Areas of Improvement

- Validate and sanitize user inputs.
- Implement JWT authentication.
- Configure Helmet.js for secure HTTP headers.
- Use parameterized SQL queries.
- Disable the `X-Powered-By` header.
- Use HTTPS in production.